REPORTE DE PRÁCTICA NO. TRES

Construcción de un Automata Finito Determinista

ALUMNO: Palma Velazquez Ariadna Betsabe

Dr. Eduardo Cornejo-Velázquez

1. Introducción

El presente documento detalla el diseño e implementación de un autómata finito determinista (AFD) desarrollado en lenguaje C++. El sistema ha sido estructurado para procesar lenguajes regulares mediante la lectura de un archivo de configuración externo en formato .txt. Este archivo define el alfabeto de entrada, permitiendo que el programa valide y filtre cadenas de caracteres con precisión. Como resultado final, el software discrimina las palabras pertenecientes al lenguaje y presenta a través de la interfaz de consola aquellas que cumplen con las reglas de transición establecidas, demostrando la aplicación práctica de la teoría de la computación en la resolución de problemas lógicos.

- ***Objetivo General:***

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de símbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

- ***Objetivos Especificos:***

1. Identificar los componentes de un Autómata Finito Determinista (AFD).

1. Implementar un AFD en un lenguaje de programación de alto nivel.

1. Determinar si diferentes conjunto de símbolos (W) pertenece al lenguaje (L) reconocido por el AFD.

2. Marco teórico

Los Autómatas Finitos son máquinas teóricas que van cambiando de estado dependiendo de la entrada que reciben. La salida se limita a dos valores: aceptado y no aceptado, para indicar si la cadena que se ha recibido como entrada es o no válida.
Un Autómata Finito Determinista (AFD) se define como una tupla:

$$\mathbf{AFD} = (\boldsymbol{\Sigma},\ \mathbf{Q},\ \boldsymbol{\delta},\ \mathbf{q_0},\ \mathbf{F})$$

- $\boldsymbol{\Sigma}$ es el alfabeto de entrada
- $\mathbf{Q}$ es el conjunto finito y no vacío de los estados
- $\boldsymbol{\delta}$ es la función de transición, $\boldsymbol{\delta}: \mathbf{Q} \times \boldsymbol{\Sigma} \rightarrow \mathbf{Q}$
- $q_0 \in Q$ es el estado inicial
- $\mathbf{F} \subset \mathbf{Q}$ es un conjunto de estado finales de aceptación $(\mathbf{F} \neq \emptyset)$

Sea $A = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$ un AFD y sea $w = w_1 w_2 \ldots w_n$ una cadena de símbolos donde $w_i \in \Sigma$.
Entonces, **A *acepta w*** si existe una secuencia de estados $r_0, r_1, \ldots, r_n \in Q$ con tres condiciones:

1. $r_o = q_0$
2. $\delta(r_i, w_i + 1) = r_i + 1\ para\ i = 0,1, \ldots, n-1$
3. $r_n \in F$

La condición 1 establece que la máquina comienza en el estado inicial.
La condición 2 establece que la máquina cambia desde un estado hacia otro estado de acuerdo a la función de transición.
La condición 3 establece que la máquina acepta la cadena de entrada si termina en un estado de aceptación.
Entonces, **A *reconoce el lenguaje* L** si $L = \{w | A\ acepta\ w\}$.

3. Herramientas empleadas

- Equipo de cómputo
- Sotware Code Blocks para C++
- Repositorio en GitHub

4. Desarrollo

Para llevar acabo esta resolucion del problema se desarrollo un codigo de alrededor de 85 lineas de codigo. Como primera parte se designan las liberias que se utilizaran, las cuales son:

- include ¡iostream¿
- include ¡string¿
- include ¡cctype¿

Despues se le solicita al usuario ingresar la cantidad que se le tiene que designar al array en el que se guarda cada uno de los datos, con los siguientes comandos:

- int N, S, D, q0, T, C;

cout ¡¡ "Ingrese el numero de estados (N): "; cin ¿¿ N; cout ¡¡ "Ingrese la cantidad de simbolos del alfabeto(S): "; cin ¿¿ S; cout ¡¡ "Ingrese el numero de transiciones (D): "; cin ¿¿ D; cout ¡¡ "Ingrese el estado inicial (q0): "; cin ¿¿ q0; cout ¡¡ "Ingrese la cantidad de estados de aceptacion(T): "; cin ¿¿ T; cout ¡¡ "Ingrese la cantidad de cadenas a evaluar (C):"; cin ¿¿ C;
Posteriormente se pide ahora si cada uno de los datos de las transiciones y el alfabeto; y los demas datos en cada uno de sus arrays. Con sus especificaciones cada uno:

- char alfabeto[10]; for (int i = 0; i ¡ S; i++) char simbolo; cout ¡¡ "Ingrese alfabeto " ¡¡ i + 1 ¡¡ " de " ¡¡ S ¡¡ ": "; cin ¿¿ simbolo; while (!isalpha(simbolo) !isdigit(simbolo)) cout ¡¡ "ERROR: Solo se permiten letras o numeros."; cout ¡¡ "Ingrese nuevamente alfabeto " ¡¡ i + 1 ¡¡ ": "; cin ¿¿ simbolo; alfabeto[i] = simbolo;
- int aceptacion[3]; for (int i = 0; i ¡ T; i++) cout ¡¡ "Ingrese estado de aceptacion " ¡¡ i + 1 ¡¡ " de " ¡¡ T ¡¡ ": "; cin ¿¿ aceptacion[i];
- cout ¡¡ "Formato: Primero Ingresar Origen luego el simbolo que entra y por ultimo el destino al que lleva"; cout ¡¡ "Debe ingresar " ¡¡ D ¡¡ " transiciones"; int origen[15]; char simbolo[15]; int destino[15]; for (int i = 0; i ¡ D; i++) cout ¡¡ "Transicion " ¡¡ i + 1 ¡¡ " de " ¡¡ D ¡¡ ": "; cin ¿¿ origen[i] ¿¿ simbolo[i] ¿¿ destino[i];
- string cadenas[10]; for (int i = 0; i ¡ C; i++) cout ¡¡ "Ingrese cadena " ¡¡ i + 1 ¡¡ " de " ¡¡ C ¡¡ ": "; cin ¿¿ cadenas[i];

Despues para poder impñrimir los resultados se hace una comparacion de cada uno de los arrays dentro de for´s y if´s que designan el funcionamiento de autoata para designar si es aceptado o rechazado la cadena dada a estudisar:

- cout ¡¡ "— RESULTADOS —"; for (int i = 0; i ¡ C; i++) int estadoActual = q0; for (int j = 0; j ¡ cadenas[i].length(); j++) char letra = cadenas[i][j]; for (int k = 0; k ¡ D; k++) if (origen[k] == estadoActual simbolo[k] == letra) estadoActual = destino[k]; break; bool aceptada = false; for (int j = 0; j ¡ T; j++) if (estadoActual == aceptacion[j]) aceptada = true; cout ¡¡ "Cadena ¨" ¡¡ cadenas[i] ¡¡ "¨¿ "; if (aceptada) cout ¡¡ "ACEPTADA"; else cout ¡¡ "RECHAZADA";

  return 0;

```cpp
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {

    int N, S, D, q0, T, C;
    cout << "Ingrese el numero de estados (N): ";
    cin >> N;
    cout << "Ingrese la cantidad de simbolos del alfabeto (S): ";
    cin >> S;
    cout << "Ingrese el numero de transiciones (D): ";
    cin >> D;
    cout << "Ingrese el estado inicial (q0): ";
    cin >> q0;
    cout << "Ingrese la cantidad de estados de aceptacion (T): ";
    cin >> T;
    cout << "Ingrese la cantidad de cadenas a evaluar (C): ";
    cin >> C;
    cout<<"\n\n";
```

```cpp
    char alfabeto[10];
    for (int i = 0; i < S; i++) {
        char simbolo;
        cout << "Ingrese alfabeto " << i + 1 << " de " << S << ": ";
        cin >> simbolo;
        while (!isalpha(simbolo) && !isdigit(simbolo)) {
            cout << "ERROR: Solo se permiten letras o numeros.\n";
            cout << "Ingrese nuevamente alfabeto " << i + 1 << ": ";
            cin >> simbolo;
        }
        alfabeto[i] = simbolo;
    }

    int aceptacion[3];
    for (int i = 0; i < T; i++) {
        cout << "Ingrese estado de aceptacion " << i + 1 << " de " << T << ": ";
        cin >> aceptacion[i];
    }

    cout << "Formato: Primero Ingresar Origen luego el simbolo que entra y por ultimo el destino al que llev
    cout << "Debe ingresar " << D << " transiciones\n";
```

```
    int origen[15];
    char simbolo[15];
    int destino[15];
    for (int i = 0; i < D; i++) {
        cout << "Transicion " << i + 1 << " de " << D << ": ";
        cin >> origen[i] >> simbolo[i] >> destino[i];
    }

   string cadenas[10];
    for (int i = 0; i < C; i++) {
        cout << "Ingrese cadena " << i + 1 << " de " << C << ": ";
        cin >> cadenas[i];
    }

    cout << "\n--- RESULTADOS ---\n";
    for (int i = 0; i < C; i++) {
        int estadoActual = q0;
        for (int j = 0; j < cadenas[i].length(); j++) {
            char letra = cadenas[i][j];
            for (int k = 0; k < D; k++) {
                if (origen[k] == estadoActual && simbolo[k] == letra) {
```


```
                if (origen[k] == estadoActual && simbolo[k] == letra) {
```


```
                    estadoActual = destino[k];
                    break;
                }
            }
        }
        bool aceptada = false;
        for (int j = 0; j < T; j++) {
            if (estadoActual == aceptacion[j]) {
                aceptada = true;
            }
        }
        cout << "Cadena \"" << cadenas[i] << "\" -> ";
        if (aceptada) {
            cout << "ACEPTADA\n";
        } else {
            cout << "RECHAZADA\n";
        }
    }

    return 0;
```

5. Conclusiones

En conclusión, la implementación de este Autómata Finito Determinista en C++ demuestra la viabilidad de transformar conceptos teóricos de la computación en herramientas funcionales de software. La capacidad del programa para adaptar su alfabeto mediante archivos externos subraya la importancia del diseño flexible, permitiendo que el mismo núcleo lógico procese distintos lenguajes sin necesidad de modificar el código fuente. Este ejercicio no solo valida el reconocimiento de cadenas según estados de transición, sino que también refuerza el dominio en el manejo de estructuras de datos y flujos de entrada/salida (I/O) en entornos de programación de alto rendimiento.

Referencias Bibliográficas

- Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de automátas. Editorial Alfaomega. Argentina.
- Ruiz Catalán, J. (2010). Compiladores: teoría e implementación. Editorial Alfaomega. México.
- Brookshear, J. G. (1995). Teoría de la Computación. Lenguajes formales, autómatas y complejidad. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.